Forkcast — Mobile Developer Handoff (React Native / Expo)

Audience: the mobile app developer. **Source of truth:** this repository (the shipped web app), not the original design mockups — the product has evolved substantially past them. Live reference: https://seymurhh.github.io/forkcast-live/

Contact: Seymur Hasanov · shasanov@seas.harvard.edu

---

1. Stack and non-negotiables

- **Expo (React Native) + TypeScript.** Chosen specifically so the domain logic in `lib/` is shared, not re-implemented.
- **Reuse, don't rewrite, these modules** (pure TypeScript, no DOM dependencies — they compile in React Native as-is):
  - `lib/nutrition.ts` — BMR/TDEE (Mifflin–St Jeor), goal targets, adaptive calibration, Fit Score, BMI classes, condition warnings, allergen logic. **This file is the product.** Any macro number shown on mobile must come from these functions.
  - `lib/types.ts` — all domain types.
  - `lib/format.ts` — money/date/uid helpers.
  - `data/restaurants.ts` — the restaurant/menu catalog with provenance metadata (`dataSource`, `sourceNote`).
  - `lib/cloud.ts` + `lib/supabase.ts` — Supabase data layer (supabase-js works in React Native with the Expo fetch polyfill; swap `localStorage`-based session storage for `@react-native-async-storage/async-storage` via the client's `auth.storage` option).
  - Recommended structure: extract these into `packages/core` (npm workspace) consumed by both the Next.js app and the Expo app. Until then, copy the files verbatim and diff on every release.
- **React-DOM-bound modules to re-implement as RN screens/hooks:** `lib/auth.tsx`, `lib/store.tsx`, `lib/order.tsx`, `lib/premium.tsx` — the logic inside is documented in §5; the React context pattern ports 1:1, only `localStorage` → AsyncStorage changes.

2. Backend (already built — do not invent your own)

Supabase project (Postgres + Auth + RLS). Schema: `supabase/migrations/0001_init.sql`. Setup: `docs/BACKEND_SETUP.md`.

- Auth: Supabase email/password. `user_metadata: { name, role: "customer" | "restaurant" }`.
- Tables: `profiles` (health profile jsonb), `meal_logs`, `weight_entries`, `orders`. All RLS: `auth.uid() = user_id`. Client uses the **anon key only**.
- Sync contract (mirror the web app): local-first, UI never blocks on network; on sign-in pull all (cloud wins if non-empty, else seed cloud from device); push per-mutation, fire-and-forget. See `lib/cloud.ts`.
- **AI endpoints** (`/api/analyze` photo estimation, `/api/chat` coach): these are Next.js server routes and do NOT exist on the static GitHub Pages site. For mobile they must be hosted (Vercel deploy of this repo, or ported to Supabase Edge Functions). Contract:
  - `POST /api/chat { messages: [{role, content}], context?: {…profile} }` → `{ reply, provider }`. Provider chain server-side: DataRobot gateway → Gemini → Groq. Keys live only on the server.
  - `POST /api/analyze` (photo, multipart/base64) → per-dish estimates with explicit `confidence`, plus `failures[]` naming any provider that errored. Nothing fails silently.

3. Screen inventory (21 routes, all shipped)

Consumer loop (the core mobile scope, in flow order):

#	Route	Screen	Mobile notes
1	`/`	Landing	Mobile: collapse into onboarding/marketing carousel
2	`/signup`, `/login`, `/forgot-password`	Auth (diner + restaurant roles)	Supabase auth; email confirm flow
3	`/onboarding`	Health profile intake (sex, age, height/weight cm/kg toggle, activity, goal, dietary, allergens, conditions)	Multi-step wizard
4	`/dashboard`	Today: calorie/macro budgets, charts, streak, calibration status, habit-based recommendations	Primary tab
5	`/discover`	Restaurant/dish search: geolocation distances, Filters button + removable pills, dish-name search, Fit-Score sort, map/list	Primary tab; use native geolocation
6	`/restaurant/[slug]`	Menu with per-dish Fit Scores, provenance badges	
7	`/restaurant/[slug]/dish/[id]`	Dish detail: macros vs remaining budget, portion slider + donut, warnings	
8	`/basket`	Basket: one restaurant per basket, substitution notes	
9	`/checkout`	Pickup/delivery, MA meals tax 7%, delivery fee $5.99, prototype-integration labeling	
10	`/order`	Live status (sent→accepted→preparing→ready), “Log this meal?” confirmation with portion edit	Push notification hook point
11	`/orders`	Order history with evidence trail	
12	`/log`	Daily log: order-confirmed, photo (AI), manual entries; per-entry provenance + confidence	Camera integration
13	`/profile`	Profile editing, membership card, role-split	
14	`/community`	Community	v2 for mobile

#	Route	Screen	Mobile notes
—	CoachChat (floating, diner-only)	AI coach: general vs personalized modes, trial/premium gating	Chat screen or sheet

Restaurant-side (`/partner` terminal) and marketing pages (`/how-it-works`, `/for-restaurants`, `/business`, `/impact`) stay web-only for v1.

4. Design tokens (Modernist system — copy exactly)

- Font: **Archivo** (Google Fonts; `@expo-google-fonts/archivo`). Display and body are the same family; display uses extrabold weights.
- Colors:
  - Ink (text): `#201e1d` · Ground (background): `#f3f2f2`
  - Brand ramp: 50 `#fff2ef` · 100 `#ffe0d9` · 200 `#ffc4b8` · 300 `#ff9783` · 400/500 `#ff563c` · **600 `#ec3013` (primary)** · 700 `#ae1800` · 800 `#7c1405` · 900 `#4d170e` · 950 `#201e1d`
  - Warm neutrals: 100 `#f8f4f4` · 200 `#eae7e7` · 300 `#d7d3d3` · 400 `#bab6b6` · 500 `#9b9797` · 600 `#7d7979` · 700 `#605d5d` · 800 `#444141` · 900 `#2d2b2b`
- Shape language: pill buttons (full radius), 2xl card radius (~16dp), 2px dividers, thin borders at 5–10% black, white cards on the ground color.
- Tone: quiet surfaces, one loud accent (brand-600). Fit Score chips, provenance badges (`BadgeCheck` icon = partner-verified), simulated/prototype labels in amber.

5. Business-logic spec (must match web exactly)

Targets (`computeTargets`): Mifflin–St Jeor BMR (`10·kg + 6.25·cm − 5·age + s`, s = +5 male / −161 female) × activity factor (sedentary 1.2, light 1.375, moderate 1.55, active 1.725, very_active 1.9) = TDEE; goal adjustment −500 (lose) / +300 (gain) kcal, floored at safe minimums. Protein/fat/carb/fiber splits per the function — do not restate them here, import the file.

Adaptive calibration (`calibrateTdee`): with ~14 days of logs + 2 weigh-ins, observed TDEE = avg intake − (Δweight kg × 7700)/days; blended with formula TDEE by data-confidence weight; status surfaced to the user ("calibrating…" vs "active"). Targets then use `blendedTdee`.

Fit Score (`fitScore`, 0–100): dish macros scored against the user's *remaining* daily budget; `personalAdjust` applies allergen exclusion and condition penalties (e.g., sodium for hypertension). Warnings via `conditionWarnings` — advisory language only, never medical advice.

Order state machine: `sent → accepted → preparing → ready` (simulated timeline 8s/25s/55s, always labeled "simulated"; a claimed partner terminal overrides via the sync bus — on mobile this arrives through Supabase later, v1 keeps the simulation). Auto-log NEVER happens silently: the post-order "Log this meal?" sheet with portion edit is mandatory product behavior.

Premium (`lib/premium.tsx`): 7-day free trial from account `createdAt`; trial/free tier: 25 coach messages/day; Premium `$4.99/mo or $39.99/yr` — unlimited coach + photo AI after trial. Payments on mobile = store IAP (RevenueCat recommended); the demo "upgrade" flag pattern shows the gating surface.

Honesty constraints (product law, carries to mobile): - Every nutrition number carries provenance: partner-verified / published / estimated (±). Estimated values always show the ± label. - No live payment/restaurant integration exists: orders are labeled "prototype integration" wherever totals appear. - Kitchen status is labeled simulated unless a terminal claimed the order. - AI outputs show confidence and are user-correctable; corrections are versioned, never silent.

6. Suggested milestones

1. **M0** — Expo scaffold, shared `core` package extracted, Supabase auth + profile sync, onboarding + dashboard (read-only).
2. **M1** — Discover (geolocation, filters, Fit Scores) + restaurant/dish + basket/checkout/order loop with log confirmation.
3. **M2** — Daily log + photo AI (hosted endpoint) + coach chat + premium gating with IAP.
4. **M3** — Push notifications (order ready, log reminder), offline hardening, store submission (TestFlight / Play internal track).

Definition of done per screen: numbers identical to the web app for the same inputs (shared core makes this automatic), provenance labels present, works offline with queued sync.